

**Smt. Chandaben Mohanbhai Patel Institute of Computer Applications,
CHARUSAT**

**BCA Semester-III
CAUE302: Python Programming**

Python Built-In Functions and User-Defined Functions

1. Introduction to Python Functions

- A Function in Python is a self-contained, reusable block of code designed to perform a specific, well-defined task. Functions eliminate code redundancy, improve modularity, simplify debugging, and make programs scalable and clean.
- In Python, functions are broadly categorized into two main types:
 1. Built-In Functions: Pre-defined utilities available directly in the Python standard library (e.g., len(), type(), sum(), min(), max(), abs(), sorted()).
 2. User-Defined Functions: Custom functions defined by developers using the 'def' keyword to perform specific application logic.

2. Part A: Python Built-In Functions

Built-in functions are always accessible without requiring any import statements. They provide fundamental services for type inspection, numerical calculations, collection processing, and data conversions.

Topic 1: Type Inspection, Conversion & Utility Functions (type, len, id, print, input, range)

Built-in inspection tools like type() return the class type, len() measures sequence size, id() returns memory identity, and range() produces number sequences.

Syntax Structure:

```
# Built-in Usage
data_type = type(obj)
size = len(sequence)
mem_id = id(obj)
```

Example 1: Type Inspection and Length Verification:

```
text = 'charusat'
num = 2024
print(f'Length of "{text}":', len(text))
print(f'Type of text:', type(text))
print(f'Type of num:', type(num))
print(f'Memory Address of text:', id(text))
```

Example 2: Data Type Conversion & Range Utility:

```
str_num = '100'
int_num = int(str_num)
float_num = float(int_num)
print(f'Converted values: {int_num} ({type(int_num)}), {float_num}
({type(float_num)})')
numbers = list(range(1, 6))
print('Generated List from range():', numbers)
```

Topic 2: Aggregate, Mathematical & Sequence Utility Functions (sum, min, max, abs, round, sorted)

Functions designed to compute summary statistics, perform rounding, reorder collections, and zip/enumerate multiple sequences.

Syntax Structure:

```
total = sum(iterable)
ordered = sorted(iterable, reverse=True)
for idx, val in enumerate(iterable):
    ...
```

Complete Example 1: Mathematical & Aggregation Functions (abs, round, sum, min, max):

```
values = [-12.5, 45.8, 3.2, -99.0, 78.4]
print('Absolute Value of -12.5:', abs(-12.5))
print('Rounded 45.876 to 2 decimals:', round(45.876, 2))
print('Sum of collection:', sum(values))
print('Maximum value:', max(values))
print('Minimum value:', min(values))

names = ['Alice', 'Bob', 'Charlie']
print('Sorted Names:', sorted(names))
```

3. Part B: Python User-Defined Functions

User-defined functions are created using the 'def' keyword, followed by the function name, parameter list in parentheses, and a colon. Advantages of User-Defined Functions:

- Code Reusability: Write once, execute multiple times across different inputs.
- Modularity: Breaks massive programs into manageable, independent sub-blocks.
- Abstraction: Hides complex internal execution details behind a simple interface.
- Personalization & Clean Code: Tailors logic to business requirements while maintaining readability.

Topic 3: Defining & Calling Functions with Return Statements

A function is defined with 'def function_name(parameters):' and invoked using 'function_name(arguments)'. The 'return' statement sends calculated results back to the caller.

Syntax Structure:

```
def function_name(param1, param2):
    # instructions
    return result
```

```
# Function Calling
res = function_name(val1, val2)
```

Complete Example 1: Basic Function Definition and Calling:

```
def greet_student(studname, branch, sem):  
    print(f'Student Name = {studname}')  
    print(f'Branch      = {branch}')  
    print(f'Semester    = {sem}')
```

```
# Calling the function  
greet_student('23BCA001', 'BCA', 5)
```

Complete Example 2: Arithmetic Operations with Return Values:

```
def calculate_math(no1, no2):  
    add = no1 + no2  
    sub = no1 - no2  
    return add, sub # Returns multiple values as tuple
```

```
sum_res, sub_res = calculate_math(10, 20)  
print(f'Sum: {sum_res} | Difference: {sub_res}')
```

Topic 4: Function Argument Variations (Default, Keyword, and Arbitrary Arguments)

Python supports flexible parameter handling:

- **Default Arguments:** Assigned default values in header if argument is omitted during call.
- **Keyword Arguments:** Passed using `parameter_name=value`, allowing arguments in any order.
- **Positional Arbitrary (*args):** Accepts variable number of positional arguments into a tuple.
- **Keyword Arbitrary (**kwargs):** Accepts variable number of keyword arguments into a dictionary.

Syntax Structure:

```
# Default  
def f1(a, b=5): pass  
  
# Arbitrary Positional (*args)
```

```
def f2(*args): pass
```

```
# Arbitrary Keyword (**kwargs)
```

```
def f3(**kwargs): pass
```

Complete Example 1: Default Arguments & Keyword Arguments:

```
def display_info(studname, branch='BCA', sem=5):  
    print(f'Name: {studname} | Branch: {branch} | Semester: {sem}')
```

```
# Using Default Arguments
```

```
display_info('23BCA001')
```

```
# Overriding Default Arguments
```

```
display_info('23BCA002', 'BCA', 6)
```

```
# Using Keyword Arguments (Order doesn't matter)
```

```
display_info(sem=3, studname='23BCA003', branch='BCA')
```

*Complete Example 2: Arbitrary Positional Arguments (*args):*

```
def calculate_total(*nums):  
    print(f'nums = {nums} | type = {type(nums)}')  
    total = sum(nums)  
    return total
```

```
print('Sum 1:', calculate_total(10, 20))
```

```
print('Sum 2:', calculate_total(10, 20, 30, 40, 50))
```

*Complete Example 3: Arbitrary Keyword Arguments (**kwargs):*

```
def print_student_profile(**details):  
    print(f'details = {details} | type = {type(details)}')  
    for key, value in details.items():  
        print(f'{key} = {value}')
```

```
print_student_profile(sid='23BCA001', branch='BCA', city='Anand', marks=92)
```